ReQuest-SQL: Metinden SQL'e Çeviride Doğrudan Şema Bağlama İçin Gelişmiş Soru Zenginleştirme Stratejileri

Yiğit Kemal Can
Bilkent Üniversitesi
Ankara, Türkiye
y.can@ug.bilkent.edu.tr

Hasan Alp Caferoğlu
Bilkent Üniversitesi
Ankara, Türkiye
alp.caferoglu@bilkent.edu.tr

Özgür Ulusoy
Bilkent Üniversitesi
Ankara, Türkiye
oulusoy@cs.bilkent.edu.tr

***Özet*—Doğal dil sorularını SQL sorgularına çevirmek (*Metinden SQL'e*), uzman olmayan kullanıcıların veritabanlarıyla günlük dili kullanarak etkileşime girmesini sağlar. Büyük dil modelleri (LLM'ler) ile sağlanan son ilerlemeler Metinden SQL'e performansını artırmış olsa da karmaşık şemaların ele alınması, belirsiz sorgular ve çok adımlı akıl yürütme için doğru SQL üretilmesi gibi zorluklar devam etmektedir. Önceki çalışmalar, şema bağlamayı geliştirmek için doğal soruyu veritabanı şeması bağlamıyla zenginleştirme fikrini ortaya atmıştır. Ancak mevcut zenginleştirme teknikleri genellikle tek geçişte tek bir geliştirilmiş soru üretmekte ve alternatif veya yinelemeli zenginleştirme stratejilerinin potansiyel faydalarını göz ardı etmektedir.**

Bu çalışmada, kullanıcı amacı ile veritabanı yapısı arasındaki boşluğu kapatmak için birden fazla soru zenginleştirme stratejisini sistematik olarak inceleyen gelişmiş bir Metinden SQL'e ardışık düzeni (pipeline) olan ReQuest-SQL'i sunuyoruz. Özel olarak, (i) bir sorunun birden çok zenginleştirilmiş sürümünü oluşturmayı ve bunların bilgilerini *birleştirmeyi*, (ii) birden çok zenginleştirilmiş soru oluşturmayı ve daha fazla iyileştirme için en iyisini *seçmeyi* ve (iii) çok turlu iyileştirme yoluyla *yinelemeli* soru zenginleştirmesi gerçekleştirmeyi öneriyoruz. Bu yöntemler, çoklu zenginleştirme girişimlerinden elde edilen tamamlayıcı şema bilgilerinden yararlanmayı ve SQL üretimi sırasında LLM'i daha etkili bir şekilde yönlendirmeyi amaçlamaktadır. Gelişmiş ardışık düzenimiz, önceki sistemlerin aşağı akış (downstream) SQL üretimi ve iyileştirme aşamalarını korurken, tek geçişli zenginleştirme adımını önerilen çok yönlü stratejilerle değiştirmektedir.

Buna ek olarak, bir çoklu aday SQL iyileştirme ve seçme mekanizması entegre ediyoruz: iyileştirme modülümüz tek bir iyileştirilmiş SQL üretmek yerine, birden fazla SQL adayı örnekler, başarı/hata sinyallerini ve kısa/özet niteliğindeki sonuçları toplamak için her adayı çalıştırır ve ardından sorunsuz çalışan ve anlamsal olarak doğru sorguyu seçmek için LLM tabanlı bir yargıç kullanır. Bu, SQL üretim aşamasına sağlamlık ekleyerek soru zenginleştirmesini tamamlar.

Çapraz alanlı (cross-domain) BIRD kıyaslamaları üzerindeki ön değerlendirmeler, karmaşık sorgularda çalışma (execution) doğruluğunun arttığını göstermekte ve çeşitli zenginleştirme yaklaşımlarımızın Metinden SQL'e görevindeki etkinliğini kanıtlamaktadır.

Ek olarak, şema filtrelemenin zenginleştirme ardışık düzenimiz üzerindeki etkisine dair bir araştırma yürüttük. Orijinal çalışma, şema filtrelemenin gerekli bağlamı kaldırarak doğruluğu azalttığını öne sürmüştü. Bu hipotezi son teknoloji bir filtreleme yaklaşımıyla test etmek için Bilgi Getirici (Information Retriever - IR) ve Şema Seçici (Schema Selector - SS) aracılarını (agents) entegre ettik. Sıkı şema budamasının, tam şema bağlamını kullanmaya kıyasla doğruluğu olumsuz etkilediğini gözlemledik.

***Anahtar Kelimeler*—Metinden SQL'e; Büyük Dil Modelleri; Şema Bağlama; Soru Zenginleştirme; İstem Mühendisliği**

I. Giriş

Doğal bir dil sorusunu SQL sorgusuna çevirme görevi, teknik bilgisi olmayan kullanıcılara veritabanları için doğal bir dil arayüzü sağlamanın bir yolu olarak uzun süredir incelenmektedir. NLP (Doğal Dil İşleme) ve veritabanlarının kesişim noktasında yer alan bu Metinden SQL'e problemi, kullanıcıların SQL sözdizimini veya şema ayrıntılarını bilmeden veritabanlarını sorgulamalarına olanak tanımayı amaçlamaktadır [11]. Önemli ilerlemelere rağmen, en gelişmiş modeller bile karmaşık sorgularda hala insan seviyesindeki performansın gerisinde kalmaktadır [11]. Belirsiz soruları doğru bir şekilde anlamak, büyük veya karmaşık veritabanı şemalarını idare etmek ve kullanıcı niyetiyle tam olarak eşleşen SQL üretmek gibi zorluklar devam etmektedir.

Güçlü LLM'lerin (örn. GPT-4) ortaya çıkması, Metinden SQL'e performansını önemli ölçüde artırmıştır. Yeni ardışık düzenler, etkileyici sonuçlar elde etmek için özenle hazırlanmış istemler (prompts) ve bağlam içi öğrenme (in-context learning) ile LLM'lerden yararlanmaktadır. Örneğin, düşünce zinciri (chain-of-thought) istemciliği, modelleri ara akıl yürütme adımları üretmeye yönlendirirken [5], öztutarlılık (self-consistency) kod çözme yöntemi birden fazla çözüm yolunu birleştirerek akıl yürütme hatalarını azaltır [6]. Bununla birlikte, kullanıcının sorusunu yeniden yapılandırarak bir LLM'in akıl yürütmesini ve şema bağlamasını geliştirmek nispeten az araştırılmış bir alandır [1].

E-SQL [1], doğal soruya ilgili veritabanı şeması öğelerini (tablolar, sütunlar, değerler) ve mantıksal koşulları doğrudan enjekte eden *soru zenginleştirme yoluyla doğrudan şema bağlama* adlı yeni bir paradigma tanıtmıştır. E-SQL'in ardışık düzeni, soruyu şema bağlamını içerecek şekilde genişleterek veya yeniden ifade ederek soru ile veritabanı arasındaki boşluğu kapatır ve LLM'in doğal dili doğru SQL bileşenleriyle eşleştirmesine yardımcı olur [1]. Bu yaklaşım, aday

yüklem (predicate) artırımı ile birleştirildiğinde, karmaşık test sorgularında kayda değer doğruluk kazanımları sağlamıştır [1]. Ancak E-SQL zenginleştirilmiş soruyu tek bir adımda üretir ve daha fazla iyileştirmez; yinelemeli zenginleştirme gelecekteki bir çalışma olarak bırakılmıştır [1]. Soruları daha *kapsamlı* veya *yinelemeli* bir şekilde zenginleştirerek LLM'e daha güçlü şema bağlama ipuçları sağlanabileceğini varsayıyoruz.

Bu makalede, gelişmiş soru zenginleştirme stratejileri sunarak E-SQL üzerine inşa edilen gelişmiş bir Metinden SQL'e sistemi olan **ReQuest-SQL**'i öneriyoruz. Doğal dil sorusunu iyileştirmek için birbirini tamamlayan üç yaklaşım inceliyoruz:

- **Çoklu Zenginleştirme Birleşimi**: Bir sorunun birden fazla zenginleştirilmiş versiyonunu oluşturmak ve içeriklerini daha kapsamlı, tek bir zenginleştirilmiş soruda birleştirmek.
- **Çoklu Zenginleştirme Seçimi ve İyileştirmesi**: Birden fazla zenginleştirilmiş soru oluşturmak, en umut verici olanı seçmek ve ardından bunu ikinci bir adımda daha da iyileştirmek.
- **Yinelemeli Zenginleştirme**: Soruyu birden fazla yineleme üzerinden kademeli olarak iyileştirmek, her adımda ilgili şema ayrıntılarını ve düzeltmeleri eklemek.

ReQuest-SQL, bu yaklaşımları soru zenginleştirme modülüne entegre ederek doğrudan şema bağlamasını ve üretilen SQL'in genel kalitesini daha da artırmayı amaçlamaktadır.

Soru formülasyonunu iyileştirmenin yanı sıra, **Çoklu Aday SQL İyileştirme ve Seçimi**'ni tanıtarak SQL iyileştirme aşamasının sağlamlığını da artırıyoruz. SQL iyileştirme modülü tek bir iyileştirilmiş sorgu üretmek yerine, birden fazla SQL adayı örnekler (daha yüksek bir örnekleme sıcaklığı aracılığıyla), çalışma başarısını/hatasını, hata mesajlarını ve hafif sonuç özetlerini yakalamak için her adayı çalıştırır ve son olarak en iyi adayı seçmek için LLM tabanlı bir yargıç kullanır. Bu aşama, çoklu zenginleştirme ile aynı genel felsefeyi izler: tek denemeden (one-shot) kaynaklanan hataları azaltmak için çeşitli aday üretimlerinden ve ardından gelen birleştirme/seçme adımından yararlanmak.

Son olarak, şema filtrelemenin etkilerini yeniden gözden geçirmek için bir deney gerçekleştiriyoruz. E-SQL, filtrelemenin doğruluğu azalttığını göstermiş olsa da, daha karmaşık bir budama yönteminin farklı sonuçlar verip vermeyeceğini görmek için uzman Bilgi Getirici ve Şema Seçici aracıları [12] kullanan gelişmiş CHESS tarzı şema filtrelemesini uyguladık. Deneylerimiz, CHESS'in verimli budamasıyla bile [12], şema filtrelemenin zenginleştirme ardışık düzenimizin performansını düşürdüğünü doğruladı ve bu yaklaşım için bağlam kullanılabilirliğini en üst düzeye çıkarmanın tercih edilebilir olduğu sonucunu destekledi.

Bu çalışmanın katkıları şu şekilde özetlenebilir:

- E-SQL ardışık düzenini, bildiğimiz kadarıyla Metinden SQL'e görevinde birden fazla zenginleştirilmiş sorgu varyantını ve yinelemeli iyileştirmeyi kullanan ilk yeni soru zenginleştirme teknikleriyle genişletiyoruz.
- Soru zenginleştirme için üç strateji uygulayıp (birleşim, seçici iyileştirme ve yinelemeli) karşılaştırıyoruz ve her birinin şema bağlamayı nasıl iyileştirdiğini ayrıntılandırıyoruz.
- Birden fazla iyileştirilmiş SQL sorgusu üreten, sinyalleri toplamak için bunları çalıştıran ve en iyi adayı seçmek için bir LLM yargıcı kullanarak tek örneklem hatalarına karşı sağlamlığı artıran çoklu aday SQL iyileştirme ve seçme mekanizmasını sunuyoruz.
- CHESS çerçevesinden aracıları entegre ederek gelişmiş şema filtrelemenin etkisini ampirik olarak araştırıyor ve katı şema budamanın, tam şema bağlamını kullanmaya kıyasla zenginleştirme tabanlı ardışık düzenlerde performansı engelleyebileceğine dair daha fazla kanıt sağlıyoruz.
- BIRD veri seti üzerinde ön deneyler yürütüyor ve gelişmiş zenginleştirme stratejilerimizin, özellikle karmaşık çok tablolu sorgularda çalıştırma doğruluğunda iyileştirmeler sağladığını gözlemliyor, böylece önerilen yaklaşımların etkinliğini doğruluyoruz.

II. İLGILI ÇALIŞMALAR

A. Büyük Dil Modelleri ile Metinden SQL'e

Büyük dil modellerinin (LLM'ler) tanıtılması Metinden SQL'e görevini temelden dönüştürdü. GPT-3, Codex ve GPT-4 gibi modeller, uygun istemlerle (prompts), göreve özgü ince ayar (fine-tuning) yapmadan doğru SQL sorguları üretebileceklerini gösterdi. Bu değişim, araştırma çabalarını özelleşmiş model mimarilerinden ziyade istem tasarımı, bağlam seçimi ve akıl yürütme stratejilerine yönlendirdi.

Yeni ardışık düzenler temel olarak istem mühendisliği ve bağlam içi öğrenme yoluyla LLM akıl yürütmesinden yararlanmaya odaklanmaktadır. *Düşünce Zinciri (Chain-of-Thought)* istemciliği [5] gibi teknikler, modeli ara akıl yürütme adımları üretmeye yönlendirirken, *öz-tutarlılık* kod çözme [6] sağlamlığı artırmak için birden fazla akıl yürütme yolunu birleştirir. Bu yöntemler, en gelişmiş Metinden SQL'e sistemlerinde (örn. DAIL-SQL [11]) yaygın olarak benimsenmiş ve kıyaslama veri setlerinde çalıştırma doğruluğunda önemli kazanımlara yol açmıştır.

Bununla birlikte, akıl yürütme odaklı istemcilik, üretilen SQL'in mantıksal tutarlılığını artırmış olsa da, girdi sorusunun kendisini yeniden formüle ederek şema bağlamayı ve soruyu anlamayı geliştirmeyi nispeten az sayıda çalışma araştırmıştır. Bu boşluk, LLM tabanlı Metinden SQL'e çerçevelerinde kullanıcı sorguları ile veritabanı şemaları arasındaki uyumu güçlendirmeyi amaçlayan çok yönlü soru zenginleştirmesi araştırmamızı motive etmektedir.

B. Şema Bağlama ve İstem Artırma

Doğal dil sorusunun doğru veritabanı şeması öğeleriyle eşleştirilmesi (*şema bağlama*), Metinden SQL'e alanında temel bir zorluktur. Geleneksel sistemler şema bağlamayı varlık çözümleyiciler (entity resolvers) gibi bileşenler aracılığıyla veya şema belirteçlerini kodlayıcıda (encoder) temsil ederek birleştirmiştir [3], [4]. Diğer bir yaygın yaklaşım, modelin odak kaybını önlemek (veya istemdeki gürültüyü azaltmak) amacıyla ilgisiz tablo ve sütunları istemden budayan *şema*

*filtrelemesi*dir. Bazı yeni yöntemler, sorgu ile ilgili şema öğelerini seçmek için öğrenilmiş sınıflandırıcılar veya LLM sezgiselleri kullanarak bir başlangıç filtreleme adımı gerçekleştirir (RESDSQL [7] veya DIN-SQL [8] örneğinde olduğu gibi). Şema filtreleme, model için görevi basitleştirebilse de gerekli bilgilerin düşmesi riskini taşır. Nitekim Maamari *ve diğerleri* [2] tarafından yakın zamanda yapılan bir çalışma, en son LLM'lerin büyük şemaları ele almada usta olduğunu ve agresif filtrelemenin performansa zarar verebileceğini bulmuştur. Bu bulgu, bir şema filtreleme modülü eklemenin gelişmiş LLM'lerde sonuçları iyileştirmediğini bildiren E-SQL çalışmasındaki [1] gözlemlerle uyumludur.

Filtrelemeye bir alternatif, modele istem içerisinde şema hakkında *daha fazla* bilgi sağlamaktır. Çeşitli çalışmalar, istemi veritabanı içeriğiyle (örnek değerler veya açıklamalar gibi) zenginleştirmenin doğruluğu artırabileceğini göstermiştir [2], [11]. E-SQL ardışık düzeni, soru zenginleştirme modülü aracılığıyla şema referanslarını doğrudan sorunun kendisine enjekte ederek bu felsefeyi benimsemektedir. Sorguları tablo ve sütun adlarını ve hatta örnek değerleri içerecek şekilde yeniden formüle ederek, model üretim sırasında doğru şema bileşenlerine yönlendirilir. Soruyu artırma (augmentation) yoluyla sağlanan bu doğrudan şema bağlaması, şema bağlama ağları veya şema grafikleri gibi önceki çabaları tamamlayan nispeten yeni bir stratejidir. Çalışmamız, soru zenginleştirmesinin birden fazla geçiş ve varyasyon yoluyla nasıl daha etkili hale getirilebileceğini araştırarak E-SQL'in bu içgörüsü üzerine inşa edilmektedir.

C. Soruyu Yeniden İfade Etme ve Zenginleştirme

Sorgu ayrıştırma (karmaşık bir soruyu alt sorulara bölme) Metinden SQL'e araştırmalarında incelenmiş olsa da [8], veritabanıyla daha iyi uyum sağlaması için tek bir soruyu yeniden formüle etmek daha az incelenmiştir. E-SQL [1], bir Metinden SQL'e ardışık düzeninde sistematik bir *soru zenginleştirme* adımı başlatan ilk çalışmadır. E-SQL, kullanıcının sorusunu ilgili tablolar, sütunlar ve koşullarla genişletmesi için LLM'e talimat veren az örnekli (few-shot) bir istem kullanır ve modelin SQL'e çevirmesini kolaylaştıran zenginleştirilmiş bir sözde soru yaratır. Bu süreç tek seferde yapılır; nihai zenginleştirilmiş soru ve ara akıl yürütme izi (SQL yapılandırma planı) birlikte çıktılanır [1].

E-SQL'in tek adımlı zenginleştirmesinin başarısı, daha fazla iyileştirmenin ek kazanımlar sağlayabileceğini düşündürmektedir. Diğer NLP görevlerinde sorguların veya istemlerin yinelemeli olarak iyileştirilmesi faydalı olmuş, hataların düzeltilmesine ve ayrıntıların kademeli olarak eklenmesine olanak tanımıştır. Örneğin, yinelemeli istemleme, metin özetlemede olgusal tutarlılığı ve bütünlüğü artırmış [9] ve diyalog üretiminde adım adım yanıt optimizasyonu için kullanılmıştır [10]. Bildiğimiz kadarıyla, yinelemeli soru zenginleştirme ve birden fazla zenginleştirilmiş sorgu adayının üretilmesi Metinden SQL'e görevine daha önce uygulanmamıştır. Çalışmamız, tek seferlik istemin ötesinde soruları zenginleştirmek için yöntemler önerip değerlendirerek bu boşluğu doldurmaktadır. Bu çalışmada, birden fazla zenginleştirilmiş sürüm oluşturarak veya çok turlu iyileştirme gerçekleştirerek, kullanıcının amacının ve şema bağlamının daha eksiksiz bir resmini yakalamayı ve böylece sonraki SQL üretimini iyileştirmeyi hedefliyoruz.

D. Çoklu Aday SQL Üretimi ve Seçimi

Soru yeniden formülasyonunun ötesinde, SQL üretim aşamasında birden fazla aday SQL sorgusu örneklenerek ve tutarlılık kontrolleri ile çalıştırma (execution) geri bildirimleri kullanılarak en iyisi seçilip sağlamlık artırılabilir. Önceki çalışmalar, birden fazla akıl yürütme yolu oluşturmanın ve bunları birleştirmenin varyansı azaltabileceğini ve tek örneklem hatalarını hafifletebileceğini göstermiştir [6]. Metinden SQL'e ardışık düzenlerinde, aday sorguların veritabanında çalıştırılarak doğrulandığı ve hataları (örn. eksik birleştirmeler veya geçersiz sütunlar) onarmak için hata mesajlarının kullanıldığı çalıştırma rehberli (execution-guided) kod çözme ve kendi kendini düzeltme (self-correction) stratejileri de yaygındır. Bu yönden motive olarak, pratik bir çoklu aday iyileştirme yaklaşımı uyguluyoruz: SQL iyileştirme modülü birkaç aday SQL sorgusu üretir, her biri başarı/hata sinyallerini ve hafif sonuç özetlerini yakalamak için çalıştırılır ve LLM tabanlı bir yargıç en doğru ve güvenilir adayı seçer. Bu tasarım, çeşitliliği ve seçimi yalnızca istem (soru) alanında değil, SQL çıktı alanında da ekleyerek çoklu zenginleştirme stratejilerimizi tamamlar.

E. Çoklu Etmenli (Multi-Agent) Şema Budama

Ana odak noktamız soru zenginleştirme olsa da, gelişmiş şema filtreleme tekniklerini de inceliyoruz. CHESS çerçevesi, büyük ölçekli şemaları işlemek için çok etmenli bir sistem önermektedir [12]. Konum Duyarlı Hashing (LSH) ve vektör veritabanlarını kullanarak ilgili varlıkları ve bağlamı getirmek için bir Bilgi Getirici (IR) etmeni kullanır. Bunu, `filter_column`, `select_tables` ve `select_columns` gibi araçları kullanarak şemayı hiyerarşik olarak budayan bir Şema Seçici (SS) etmeni izler. CHESS çok büyük şemalarda etkinlik göstermiş olsa da, bu tür hedefli budamanın zenginleştirme stratejilerimizi tamamlayıp tamamlamadığını veya engelleyip engellemediğini test etmek için bu etmenleri ardışık düzenimize entegre ettik.

III. Metodoloji

Önerilen yaklaşımımız ReQuest-SQL, üç stratejimizi (çoklu zenginleştirme birleşimi, çoklu zenginleştirme seçimi/iyileştirmesi ve yinelemeli zenginleştirme) içerecek şekilde soru zenginleştirme modülünü değiştirerek E-SQL ardışık düzenini genişletir. Genel ardışık düzen E-SQL ile aynı aşamalardan oluşur: (1) Orijinal soru ve şema verildiğinde LLM tarafından başlangıç SQL sorgusunun üretildiği **Aday SQL Üretimi (CSG)**; (2) Potansiyel değerleri çıkarmak için başlangıç SQL'ini ayrıştıran ve veritabanını arayarak ek aday yüklemler (koşullar) üreten **Aday Yüklem Üretimi (CPG)**; (3) Doğal dil sorusunun iyileştirildiği **Soru Zenginleştirme (QE)** (yeni stratejilerimizin uygulandığı yer burasıdır); ve (4) Zenginleştirilmiş soruyu, aday yüklemleri ve nihai SQL sorgusunu

üretmek için başlangıç SQL'inden gelen yürütme geri bildirimlerini kullanan **SQL İyileştirme (SR)** [1]. Aşağıda önce soru zenginleştirme aşamasında getirilen değişiklikleri, ardından çoklu aday SQL iyileştirme uzantımızı açıklıyoruz.

A. E-SQL'de Temel Soru Zenginleştirme

Orijinal E-SQL'de, soru zenginleştirme modülü kullanıcının sorusunu ve ilgili şema öğelerinin listesini (CPG tarafından belirlenenler dahil) alır ve LLM'i sorunun zenginleştirilmiş bir versiyonunu üretmeye yönlendirir [1]. İstem, tabloların, sütunların ve değerlerin bir soruya nasıl entegre edileceğine dair birkaç örnek (few-shot) ile SQL akıl yürütme adımlarının bir taslağını üretecek şekilde oluşturulur. Bu istem verildiğinde LLM, gereken şema bileşenlerini açıkça belirten ve netlik için sorguyu genellikle yeniden ifade eden tek bir zenginleştirilmiş soru üretir. Örneğin, *"Lütfen 2000/1/1 tarihinden sonra açılan doğrudan charter destekli okulların telefon numaralarını listeleyin"* gibi bir kullanıcı sorusu şu şekilde zenginleştirilir: *"Lütfen charter okulu olan (`frpm.Charter School (Y/N) = 1`) ve charter fon türü doğrudan fonlanan (`frpm.Charter Funding Type = 'Directly funded'`) ve açılış tarihi 2000-01-01'den sonra olan (`schools.OpenDate > '2000-01-01'`) okulların telefon numaralarını (`schools.Phone`) bulun. `frpm` ve `schools` tablolarını `CDSCode` sütunlarını kullanarak birleştirin ve `schools` tablosundan `Phone` sütununu seçmeden önce belirtilen tüm koşulları uygulayın."*

İlk zenginleştirme adımının çıktısını temsil eden bu ilk zenginleştirilmiş soruyu $Q^{(1)}$ olarak adlandırıyoruz. $Q^{(1)}$ kullanıcının orijinal amacını açık şema ayrıntılarıyla birleştirir ve temel ardışık düzende SQL iyileştirme aşamasına girdi olarak hizmet eder.

Temel zenginleştirme işlemi tek adımda gerçekleşir: tek bir istemden tek bir zenginleştirilmiş soru elde edilir. E-SQL'in ardışık düzeninde bu zenginleştirilmiş soruyu doğrulamak veya iyileştirmek için bir mekanizma yoktur. ReQuest-SQL bu genel süreci sürdürür, ancak birden fazla zenginleştirilmiş soru üretip kullanacak ve/veya birden fazla iyileştirme turu gerçekleştirecek şekilde artırır.

B. Birleştirme (Combination) ile Çoklu Soru Zenginleştirme

İlk strateji olan **Çoklu Zenginleştirme Birleşimi**, aynı sorunun $Q_1^{(1)}, Q_2^{(1)}, \ldots, Q_n^{(1)}$ olarak gösterilen birkaç zenginleştirilmiş versiyonunu oluşturmayı ve ardından bunları özel bir *Birleştirme Modülü* aracılığıyla birleştirerek daha geniş bir ilgili bilgi yelpazesini yakalamayı amaçlar. Bu modül tüm zenginleştirilmiş soruları manuel veya sezgisel olarak birleştirmek yerine orijinal soruyu, veritabanı şemasını, sütun açıklamalarını, veritabanı örneklerini, ilgili kanıtları, olası SQL koşullarını ve gerekçeleriyle birlikte zenginleştirilmiş soru adaylarının listesini alır. Daha sonra bu adayları birlikte değerlendirir ve en güçlü yönlerini tutarlı, tek bir zenginleştirilmiş soruda bütünleştirir.

Birleştirme modülü basit bir metinsel birleştirme (union) gerçekleştirmez. Her bir adayın şemaya nasıl bağlandığını analiz eder ve hangi parçaların orijinal sorunun amacını en iyi yansıttığını belirler. Bu analize dayanarak, ek filtreler veya açıklayıcı nitelikler gibi birden fazla adaydan gelen tamamlayıcı ayrıntıları tutarlı bir sürümde birleştirir. Bu sentezin sonucu, tüm $Q_i^{(1)}$ içindeki şemayla ilgili ayrıntıları toplayan, birleşik ve geliştirilmiş zenginleştirilmiş soruyu temsil eden $Q^{(2)}$ olarak adlandırılan sorudur. Modül $Q^{(2)}$'nin veritabanı şemasına sadık kalmasını, gereksiz tekrarlardan kaçınmasını ve doğru büyük harf kullanımı ile tanımlayıcı adlarını korumasını sağlar.

Bu yaklaşımda birleştirme süreci, LLM'e zenginleştirilmiş adayları değerlendirmesi, karşılaştırması ve sentezlemesi için ayrıntılı talimatlar verilerek gerçekleştirilir. İstem, modeli açıkça sorunun en doğru ve eksiksiz yorumunu belirlemeye ve kısa bir mantıkla birlikte son bir zenginleştirilmiş sürüm üretmeye yönlendirir. Modül iki çıktı üretir: (i) entegre sürümün soru ile şema arasındaki en eksiksiz ve doğru eşlemeyi nasıl ve neden yakaladığını özetleyen birleşik akıl yürütme, ve (ii) nihai birleştirilmiş zenginleştirilmiş soru $Q^{(2)}$. Sonuç olarak nihai çıktı, ilgili şema referanslarının ve mantıksal koşulların tam kümesini kapsarken hassasiyeti ve akıcılığı koruyarak LLM'in tüm adaylara dair entegre anlayışını yansıtır.

C. Seçim (Selection) ve İyileştirme ile Çoklu Soru Zenginleştirme

İkinci strateji olan **Çoklu Zenginleştirme Seçimi ve İyileştirmesi**, çoklu zenginleştirilmiş soru adaylarını $Q_1^{(1)}, Q_2^{(1)}, \ldots, Q_n^{(1)}$ değerlendiren ve sonraki SQL üretimi için en uygun olanı belirleyen özel bir *Seçim Modülü* sunar. Bu modül orijinal soruyu, veritabanı şemasını, sütun açıklamalarını, temsili veritabanı örneklerini, ilgili kanıtları, olası SQL koşullarını ve her biri zenginleştirme mantığıyla birlikte gelen bir dizi zenginleştirilmiş soru adayını alır. Amacı, tüm adaylardan içerik birleştirmek değil, şemayla uyumu korurken orijinal sorunun amacını aslına en uygun ve eksiksiz şekilde yakalayan tek adayı belirlemektir.

Seçim modülü yapılandırılmış bir değerlendirme süreci yürütür. İlk olarak referans alınan tüm tabloların, sütunların ve ilişkilerin geçerli ve alakalı olup olmadığını kontrol ederek her adayın şemayla yazışmasını inceler. Ardından, eksik koşullardan ve gereksiz kısıtlamalardan kaçınarak her adayın kullanıcının orijinal sorusunun amacını ne kadar iyi ifade ettiğini değerlendirir. Şema örnekleri, kanıtlar ve potansiyel SQL yüklemleri ile çapraz başvuru yaparak, nihai seçilmiş zenginleştirilmiş sürüm olarak hizmet veren ve $Q^{(2)}$ olarak adlandırılan en iyi zenginleştirilmiş soruyu belirler. Büyük harf kullanımını düzeltmek, yanlış adlandırılmış bir sütunu düzeltmek veya fazlalıkları gidermek gibi küçük yüzey düzeyinde düzeltmeler gerekliyse, düzeltilmiş bir sürüm $\widetilde{Q}^{(2)}$ üretmek için bunlar doğrudan modül içinde uygulanır.

Modül daha sonra iki öğe çıkarır: (i) son, muhtemelen iyileştirilmiş zenginleştirilmiş soru ($Q^{(2)}$ veya $\widetilde{Q}^{(2)}$) ve (ii) hiçbir değişiklik yapılmadan korunan, seçilen adayın orijinal zenginleştirme akıl yürütmesi. Bu yaklaşım ReQuest-SQL'in kısa, aslına sadık ve şema açısından doğru istemleri korumasını sağlar.

D. Yinelemeli (Iterative) Soru Zenginleştirme

Üçüncü strateji olan **Yinelemeli Zenginleştirme**, önceden zenginleştirilmiş bir soruyu değerlendirmek ve gerekirse daha da geliştirmek için tasarlanmış *Yinelemeli Zenginleştirici Modül* adlı yeni bir bileşen sunar. Temel zenginleştirme veya seçim stratejilerinin aksine bu modül, ilk zenginleştirme adımı standart soru zenginleştirme istemi tarafından tamamlandıktan sonra çalışır. Orijinal kullanıcı sorusunu ve zenginleştirme mantığıyla birlikte ilk zenginleştirilmiş halini ($Q^{(1)}$), şema, sütun açıklamaları, veritabanı örnekleri, kanıtlar ve olası SQL koşullarını alır ve ek bir iyileştirme gerekip gerekmediğini belirler.

Modül yapılandırılmış bir değerlendirme prosedürünü izler. Öncelikle $Q^{(1)}$'in soru ile şema arasındaki mantıksal ve yapısal eşlemeyi ne kadar iyi yakaladığını analiz eder. Başvurulan tüm tabloların, sütunların ve yüklemlerin geçerliliğini veritabanı yapısına karşı kontrol eder ve bunların sağlanan açıklamalar ve örnek değerlerle uyumlu olduğunu doğrular. Ayrıca zenginleştirme mantığının bu bağlantıları yeterince gerekçelendirip gerekçelendirmediğini denetler. $Q^{(1)}$ zaten tam, kesin ve şemaya uygunsa, modül onu optimal olarak tanımlar ve hiçbir değişiklik yapmadan doğrudan döndürür. Bu durumda modülün akıl yürütme çıktısı, örneğin tüm şema ilişkileri ve yüklemleri kapsandığı için veya hiçbir belirsiz ya da eksik bağlantı tespit edilmediği için neden daha fazla zenginleştirmeye gerek olmadığını açıkça izah eder.

İyileştirme gerektiğinde modül, güncellenmiş bir sürüm $Q^{(2)}$ üretmek için soruyu iyileştirir. Bu iyileştirme eşleşmeyen veya yanlış yazılmış tablo/sütun adlarının düzeltilmesini, şema terminolojisini kullanarak belirsiz ifadelerin netleştirilmesini veya filtrelerin gerçek veritabanı değerleriyle veya olası SQL koşullarıyla daha iyi eşleşecek şekilde ayarlanmasını içerebilir. İyileştirme asgari düzeydedir ve her zaman şemaya ve kanıtlara dayandırılır; gereksiz yeniden anlamlandırma (paraphrasing) veya yalnızca biçimsel olan yeniden yazımlardan kesinlikle kaçınılır. Her değişiklik, önceki zenginleştirmeye kıyasla neyin iyileştirildiğini ve değişikliğin şema hizalamasını ve mantıksal tutarlılığı neden güçlendirdiğini açıklayan güncellenmiş bir akıl yürütme izi aracılığıyla belgelenir.

Modül şunları çıktılar: (i) iyileştirilmiş sürüm $Q^{(2)}$ veya herhangi bir değişiklik gerekmediyse orijinal $Q^{(1)}$ olmak üzere son zenginleştirilmiş soru ve (ii) önceki zenginleştirmenin iyileştirilip iyileştirilmediğini veya zaten optimal olup olmadığını açıklayan zenginleştirme akıl yürütmesi.

Yinelemeli zenginleştirici böylece zenginleştirme kalite kontrolü için hedeflenmiş bir son işlemci (post-processor) görevi görür. SQL İyileştirme aşamasına giren zenginleştirilmiş her sorunun optimal olarak onaylanmasını veya şemaya daha uygun ve açık bir sürüme geliştirilmesini sağlar. Pratikte bu modül, küçük isimlendirme uyumsuzlukları veya eksik mantıksal bağlantılar gibi ufak zenginleştirme hatalarının aşağı akışa (downstream) yayılmasını önlemeye yardımcı olur. Yinelemeli Zenginleştirici, zenginleştirilmiş soruyu kontrollü ve açıklanabilir bir şekilde değerlendirerek ve gerektiğinde geliştirerek soru zenginleştirme süreci için güvenilir bir nihai doğrulama adımı sağlar.

E. SQL İyileştirme ve Çalıştırma

Soru zenginleştirme çalışmamıza ek olarak, aşağı akış SQL üretim aşaması için sağlamlık sağlayan bir uzantıyı da araştırdık: **Çoklu Aday SQL İyileştirme ve Seçimi**. Önemli bir nokta, bu deneyin *standart E-SQL soru zenginleştirme ardışık düzeni* (yani tek bir geçişte üretilen tek bir zenginleştirilmiş soru) izlenerek gerçekleştirilmiş olmasıdır, dolayısıyla birden fazla zenginleştirilmiş soru üretmeyi veya birleştirmeyi içermez.

Temel QE adımı aracılığıyla zenginleştirilmiş soruyu elde ettikten sonra, bunu CPG'den gelen aday yüklemler ve CSG'den gelen ilk SQL ile birlikte SQL İyileştirme (SR) modülüne sağlarız. Temel E-SQL ayarında SR, zenginleştirilmiş soruyu ve başlangıç SQL'ini çalıştırmadan kaynaklanan herhangi bir hata mesajını kullanarak tek bir iyileştirilmiş SQL sorgusu üretir [1]. Bizim sistemimizde SR'ı, örnekleme yapılandırmasını ayarlayarak (yani bu modül için sıcaklığı artırarak) ve birden fazla çıktı isteyerek **üç** adet iyileştirilmiş SQL adayı üretecek şekilde değiştiriyoruz. Bu adaylar, her birine kısa bir akıl yürütme izinin eşlik ettiği S_1, S_2, S_3 olsun. Şunları yakalamak için her bir aday **hemen veritabanında çalıştırılır**:

- **Çalıştırma başarısı/hatası** (sorgunun hatasız çalışıp çalışmadığı),
- **Hata mesajları** (varsa),
- Satır/sütun sayıları ve (varsa) küçük örnek çıktılar gibi **sonuç özetleri**.

Daha sonra orijinal doğal dil sorusunu, şemayı, veritabanı örneklerini ve açıklamalarını, kanıtları ve çalışma (execution) sinyalleriyle birlikte üç SQL adayını alan **yargıç olarak LLM (LLM-as-a-judge) Seçim Modülü**nü sunuyoruz. Yargıç soruya göre doğruluk, çalıştırma başarısı, mantıksal tutarlılık (birleştirmeler/filtreler/toplamalar), bütünlük ve verimlilik temelinde en iyi adayı seçer. Çıktı, değerlendirme ve çalıştırma doğruluğu hesaplaması için nihai sorgu haline gelen seçilen SQL $S^\star \in \{S_1, S_2, S_3\}$'dır.

Bu çoklu aday SR mekanizması, çoklu geçiş soru zenginleştirme stratejilerimizden *ayrı bir eksen* olarak değerlendirilir. Bu nedenle, aksi açıkça belirtilmedikçe, QE-Birleşim, QE-Seçim ve Yinelemeli Zenginleştirme için rapor edilen sonuçlar ilgili zenginleştirme modüllerine karşılık gelirken, çoklu SQL iyileştirme ve seçim deneyi gelişmiş bir SR aşamasına sahip temel QE ayarına karşılık gelir.

F. CHESS Tarzı Şema Filtreleme

Yukarıda açıklanan zenginleştirme stratejilerinden bağımsız olarak, şema filtrelemenin ardışık düzenin performansı üzerindeki etkisini doğrulamak için bir deney gerçekleştirdik. Orijinal E-SQL çalışması, şema filtrelemenin modeli bağlamdan mahrum bırakarak doğruluğu azalttığını iddia etmiştir. Bu hipotezi daha gelişmiş bir yaklaşımla test etmek için CHESS çerçevesindeki aracıları entegre ettik [12].

CHESS **Bilgi Getirici (Information Retriever - IR)** ve **Şema Seçici (Schema Selector - SS)** aracılarını bağımsız bir aşama olarak uyguladık. Bu aracılar şu şekilde çalışır:

- **Bilgi Getirici (IR):** Bu etmen ilgili veri değerlerini ve bağlamını çıkarır [12]. Sorudaki anahtar kelimelerle (yazım hataları olsa bile) eşleşen veritabanı değerlerini verimli bir şekilde almak için Konum Duyarlı Hashing (LSH) kullanır ve anlamsal olarak ilgili sütun açıklamalarını getirmek için bir vektör veritabanı kullanır.
- **Şema Seçici (SS):** Bu etmen şemayı yönetilebilir bir alt kümeye budar [12]. Hiyerarşik bir süreç kullanır: önce bir LLM kullanarak sütunları filtreler (`filter_column`), sonra ilgili tabloları seçer (`select_tables`) ve son olarak sütun seçimini hassaslaştırır (`select_columns`).

E-SQL'in orijinal bulgularıyla tutarlı olarak, bu CHESS tarzı filtrelemeyi kullanan deneylerimiz, akıllı aracılar aracılığıyla bile olsa şema bağlamını sınırlamanın ardışık düzenimiz için daha düşük çalıştırma doğruluğuna yol açtığını gösterdi. Bu durum zenginleştirme modülünün, doğru bağlantı kurmak için daha geniş şema bağlamına erişimden fayda sağladığını doğrulamaktadır.

IV. Deneyler ve Sonuçlar

Tüm ardışık düzen aşamaları için temel LLM olarak Gemini-2.5-Flash modelini kullanarak ReQuest-SQL'in önerdiği soru zenginleştirme stratejilerinin etkinliğini **tam BIRD geliştirme seti** [13] üzerinde değerlendirdik. Ana değerlendirme metriğimiz, üretilen SQL sorgularından veritabanında çalıştırıldığında doğru sonuç kümesini döndürenlerin yüzdesini ölçen **Çalıştırma Doğruluğu (EX)**'dur. Önerilen üç zenginleştirme yöntemimizi (**Çoklu Zenginleştirme Birleşimi**, **Çoklu Zenginleştirme Seçimi** ve **Yinelemeli Zenginleştirme**), E-SQL'de sunulan temel tek geçişli soru zenginleştirme yaklaşımıyla karşılaştırıyoruz.

Tablo I, önerilen zenginleştirme stratejilerimizin tam geliştirme seti üzerindeki karşılaştırmalı sonuçlarını sunmaktadır. Temel zenginleştirme %64,90'lık bir çalıştırma doğruluğu elde etti. Önerilen üç stratejimizin tümü bu temele göre iyileştirmeler sağlayarak, soru zenginleştirme adımını yeniden gözden geçirip iyileştirmenin daha iyi aşağı akış SQL üretimine yol açtığı hipotezini doğrulamıştır. **Çoklu Zenginleştirme Birleşimi** stratejisi mütevazı bir iyileşme göstererek %65,12'ye ulaşmıştır. Bu da adayları birleştirmenin şemayla ilgili ek bilgileri yakalarken bazen gereksizlikler ortaya çıkarabileceğini düşündürmektedir. **Çoklu Zenginleştirme Seçimi** stratejisi %65,37 ile daha iyi performans göstererek yüksek kaliteli tek bir zenginleştirilmiş soru belirlemenin genellikle birden fazla alternatifi birleştirmekten daha etkili olduğunu göstermiştir.

Zenginleştirme stratejileri arasındaki en yüksek performans, %65,70 çalıştırma doğruluğuna ulaşan **Yinelemeli Zenginleştirici** tarafından elde edilmiştir. Bu sonuç, modelin SQL üretiminden önce küçük şema bağlantı hatalarını düzeltmek ve belirsiz mantığı açıklığa kavuşturmak için kendi zenginleştirmesini değerlendirdiği ve geliştirdiği tahsis edilmiş bir zenginleştirme sonrası kalite kontrol adımının etkinliğini vurgulamaktadır.

TABLE I
ReQuest-SQL Zenginleştirme Stratejilerinin Çalıştırma Doğruluğu (BIRD Geliştirme Seti)

Yöntem	Model	EX Doğruluğu (%)
Temel Zenginleştirme	Gemini-2.5-Flash	64.66
QE-Birleşim	Gemini-2.5-Flash	65.12
QE-Seçim	Gemini-2.5-Flash	65.37
Yinelemeli Zenginleştirici	Gemini-2.5-Flash	65.70

Soru zenginleştirmesine ek olarak, tek başına **çoklu aday SQL iyileştirme ve seçimi**nin etkisini değerlendirmek için ayrı bir deney yürüttük. Bu karşılaştırmada, E-SQL'in *temel tek geçişli soru zenginleştirme ardışık düzenini* izledik ve yalnızca SQL iyileştirme aşamasını değiştirdik. Spesifik olarak standart tek SQL'li iyileştirme ayarını, SQL iyileştirme modülünün birden fazla aday SQL sorgusu ürettiği ve yürütme geri bildirimi ile LLM tabanlı bir yargıç kullanarak en iyisini seçtiği önerilen yaklaşımımızla karşılaştırdık.

Her iki yapılandırma da tamamen aynı istemler, şemalar ve model ayarları kullanılarak **BIRD geliştirme seti** üzerinde değerlendirildi. İki ayar arasındaki tek fark, SQL iyileştirme aşamasının tek bir sorgu mu yoksa seçim izleyen birden fazla aday mı çıkardığıdır.

TABLE II
Çoklu Aday SQL İyileştirmenin Etkisi (BIRD Geliştirme Seti)

SQL İyileştirme Stratejisi	Model	EX Doğruluğu (%)
Tek SQL Üretimi	Gemini-2.5-Flash	64.66
Çoklu SQL Üretimi	Gemini-2.5-Flash	65.51

Sonuçlar, birden fazla iyileştirilmiş SQL adayını örneklemenin ve çalıştırma sonuçları ile mantıksal tutarlılığa dayalı olarak en iyisini seçmenin, tek seferlik SQL iyileştirmesine göre bir iyileşme sağladığını göstermektedir. Bu kazanım soru zenginleştirme aşamasını değiştirmeden elde edilmiştir, bu da çoklu aday SQL seçiminin ardışık düzenin diğer bileşenlerinden bağımsız, ek bir dayanıklılık (robustness) katkısı sağladığını göstermektedir.

Son olarak gelişmiş şema filtrelemenin zenginleştirme ardışık düzeni üzerindeki etkisini araştırdık. Bu analiz için şema budamasının etkilerini izole etmek amacıyla geliştirme verilerinin daha küçük bir alt kümesi olan **BIRD minidev** bölümünü kullandık. Tablo III, tam şema erişimine sahip ardışık düzenin performansını CHESS tarzı Bilgi Getirici ve Şema Seçici aracılarıyla artırılmış ardışık düzen ile karşılaştırmaktadır.

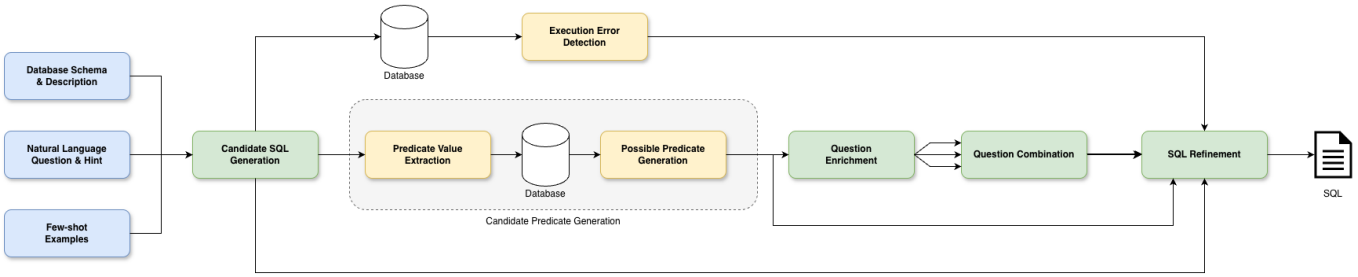


Fig. 1. QE-Birleşim ardışık düzeni. Birden fazla zenginleştirilmiş aday, şemaya sadık tek bir soruda birleştirilir.

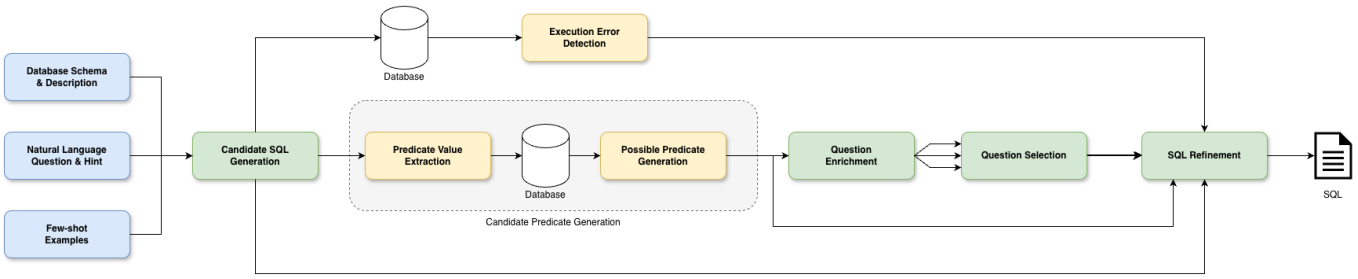


Fig. 2. QE-Seçim ve iyileştirme ardışık düzeni. En iyi aday seçilir ve gerekirse hafifçe düzeltilir.

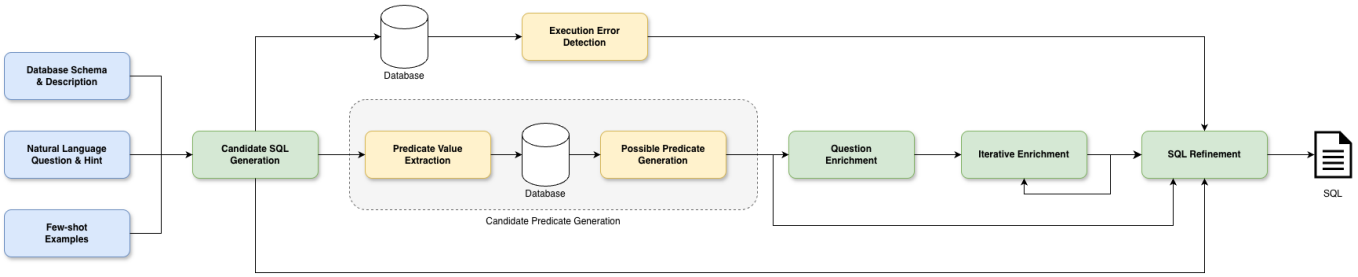


Fig. 3. Yinelemeli Zenginleştirici ardışık düzeni. Asgari düzeyde ve şemaya dayalı düzenlemelerle zenginleştirme sonrası kalite kontrolü.

TABLE III
ŞEMA FİLTRELEMENİN ÇALIŞTIRMA DOĞRULUĞU (BIRD MINIDEV SETI)

Yöntem	Model	EX Doğruluğu (%)
Temel Zenginleştirme	Gemini-2.5-Flash	63.20
Şema Filtreleme	Gemini-2.5-Flash	59.20

Tablo III'de gösterildiği gibi, şema filtrelemenin dahil edilmesi çalıştırma doğruluğunda %63,20'den %59,20'ye önemli bir düşüşe yol açmaktadır. Bu sonuç, modern LLM'lerin tam şema bağlamına erişimden fayda sağladığını belirten önceki bulguları desteklemektedir. Şema filtreleme istemdeki gürültüyü azaltmayı amaçlasa da, yanlışlıkla doğru soru zenginleştirme ve şema bağlama için gereken kritik şema öğelerini kaldırabilir ve nihayetinde performansa zarar verebilir.

V. SONUÇ

Bu makalede, mevcut soru zenginleştirme tekniklerini geliştiren gelişmiş bir Metinden SQL'e ardışık düzeni olan **ReQuest-SQL**'i sunduk. Çoklu geçişli stratejileri, özellikle Çoklu Zenginleştirme Birleşimi, Seçimi ve Yinelemeli Zenginleştirmeyi sistematik olarak araştırarak, ara zenginleştirilmiş sorunun kalitesinin nihai SQL doğruluğunu etkilediğini gösterdik. BIRD kıyaslaması üzerindeki deneylerimiz, her üç stratejinin de standart tek geçişli zenginleştirmeden daha iyi performans gösterdiğini ve **Yinelemeli Zenginleştirme**nin en yüksek çalıştırma doğruluğunu verdiğini göstermektedir. Bu bulgu, modelin kod üretiminden önce şema bağlama kararlarını doğrulamasına ve temellendirmesine olanak tanıyan, kendi kendini düzelten bir iyileştirme adımının değerini vurgulamaktadır.

Ek olarak, sorgu üretimi aşamasında sağlamlığı artırmak için bir **Çoklu Aday SQL İyileştirme ve Seçimi** mekanizması uyguladık. Birden fazla iyileştirilmiş SQL adayını örnekleyerek, çalışma başarısını ve veritabanı geri bildirimini gözlemlemek için her bir adayı çalıştırarak ve en iyi adayı seçmek için bir LLM yargıcı kullanarak, ardışık düzen tek seferlik hatalara karşı daha az hassas hale gelmekte ve çalıştırılamayan veya

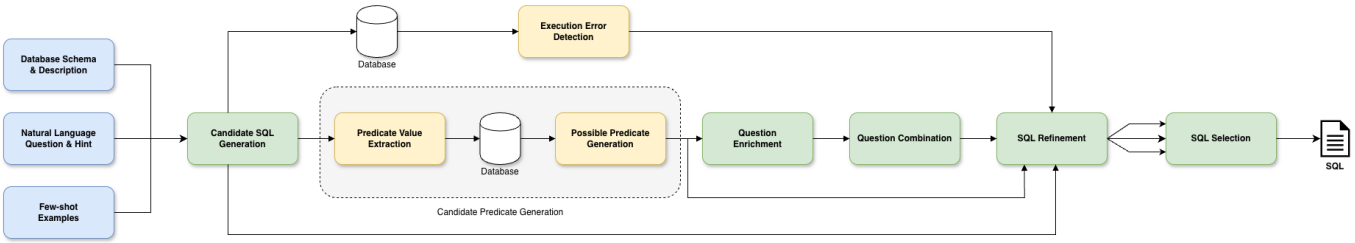


Fig. 4. Çoklu SQL üretimi ve seçim ardışık düzeni. LLM çağrısı kullanılarak en iyi aday seçilir.

mantıksal olarak kusurlu SQL'den kaçınmada daha başarılı olmaktadır. Bu bileşen, çoklu zenginleştirme stratejilerimizle aynı üst düzey prensibi izler: üretimde çeşitlilik ve ardından gelen açık bir birleştirme/seçme adımı.

Ayrıca, CHESS tarzı aracıları kullanarak şema filtrelemeye yönelik araştırmamız, veritabanı şemasını katı bir şekilde budamanın, zenginleştirme tabanlı ardışık düzenlerde performans açısından zararlı olabileceğini yeniden doğruladı. LLM'e tam şema bağlamını sağlamak, doğru soru yeniden formülasyonu için ince ilişkilerin ve daha az bariz sütunların kullanılabilir olmasını güvence altına alır.

Gelecekteki çalışmalar için, potansiyel olarak performansı en üst düzeye çıkarmak amacıyla bu stratejileri kademelendirmeyi planlıyoruz; örneğin, birden fazla seçenek arasından en iyi adayı seçmek ve ardından onu yinelemeli iyileştirmeye tabi tutmak. Ek olarak, ReQuest-SQL'i daha geniş bir yelpazedeki LLM'ler ve veri setleri üzerinde değerlendirmek, Metinden SQL'e alanında gelişmiş soru zenginleştirmesinin faydasını daha da genelleştirmeye yardımcı olacaktır. Ayrıca SQL üretim doğruluğu üzerindeki etkilerini birlikte görmek amacıyla, çok geçişli soru zenginleştirme stratejileri ile çoklu aday SQL seçiminin katkısını (örneğin, tek çıktılı SR'ye karşı yargılama işlevli çok çıktılı SR'yi karşılaştırarak) açıkça ayrıştırmayı planlıyoruz.

REFERENCES

[1] H. A. Caferoğlu and Ö. Ulusoy, "E-SQL: Direct Schema Linking via Question Enrichment in Text-to-SQL," *arXiv preprint arXiv:2409.16751*, 2024.

[2] K. Maamari, F. Abubaker, D. Jaroslawicz, and A. Mhedhbi, "The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models," *arXiv preprint arXiv:2408.07702*, 2024.

[3] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning," *arXiv preprint arXiv:1709.00103*, 2017.

[4] C. Guo, C. Sun, L. Lu, W. Ma, Q. Ye, and M. Li, "IRNet: A Lightweight Interactive Network for Text-to-SQL," in *Proc. ACL*, 2019, pp. 1935–1945.

[5] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[6] X. Wang *et al.*, "Self-Consistency Improves Chain-of-Thought Reasoning in Language Models," *arXiv preprint arXiv:2203.11171*, 2022.

[7] C. Li, W. Lu, J. Xie, Y. Zhang, and X. Chen, "RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL," in *Proc. ACL*, 2023.

[8] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," *arXiv preprint arXiv:2304.11015*, 2023.

[9] J. Chen, H. Wang, and X. Li, "Iterative Reasoning Improves Factuality in Large Language Model Summarization," *arXiv preprint arXiv:2305.12966*, 2023.

[10] A. Madaan, S. Zhang, and D. Radev, "Self-Refine: Iterative Refinement with Self-Feedback," *arXiv preprint arXiv:2303.17651*, 2023.

[11] D. Gao *et al.*, "Text-to-SQL Empowered by Large Language Models: A Benchmark Study," *Proc. VLDB Endowment*, vol. 17, no. 11, pp. 3222–3235, 2024.

[12] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual Harnessing for Efficient SQL Synthesis," *arXiv preprint arXiv:2405.16755*, 2024.

[13] J. Li *et al.*, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-scale Database Grounded Text-to-SQLs," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

EK A
İSTEM ŞABLONLARI

A. *QE-Birleşim (Combination)*

```
### Bir soru ile ilgili veritabanı arasındaki bilgileri kusursuz bir şekilde ilişkilendiren mükemmel
bir veri bilimcisisiniz. Göreviniz orijinal soruyu, veritabanı şemasını, veritabanı sütun
açıklamalarını, veritabanı örnek değerlerini, mevcut kanıtları ve olası SQL koşullarını **birden
fazla zenginleştirilmiş soru adayıyla birlikte** okumaktır. Daha sonra, bu adaylardan gelen **en iyi,
en kesin öğeleri entegre ederek** **TEK bir nihai zenginleştirilmiş soru** **üretmeli ve
birleştirilmiş sorunun ima ettiği SQL yapılandırma planını sıralayan kısa, adım adım bir birleştirme
akıl yürütmesi** oluşturmalısınız.

### Aşağıdaki talimatları izleyin (QE spesifikasyonuna uyarlanmıştır):
# Adım 1 - Soruyu Dikkatlice Okuyun: Ana amacı ve ayrıntıları anlayın.
# Adım 2 - Veritabanı Şemasını Analiz Edin: İlgili tabloları, sütunları, ilişkileri belirlemek için ş
emayı kullanın.
# Adım 3 - Veritabanı Sütun Açıklamalarını İnceleyin: Sorudaki terimleri şema öğeleriyle daha iyi
hizalamak için açıklamaları kullanın.
# Adım 4 - Veritabanı Örnek Değerlerini Analiz Edin ve Gözlemleyin: Olası değerleri/filtreleri
temellendirmek için örnekleri kullanın.
# Adım 5 - Kanıtları İnceleyin: İlgili kısımlara odaklanmak için kanıtları kullanın.
# Adım 6 - Olası SQL Koşullarını Analiz Edin: Önerilen aday yüklemleri/koşulları dikkate alın.
# Adım 7 - Zenginleştirilmiş Adaylar Arasında Sentez Yapın: Tüm adaylardan en güçlü tablo/sütun/değer
bağlantılarını ve ifadeleri birleştirin.
# Adım 8 - Tek Bir Birleştirilmiş Zenginleştirilmiş Soru Üretin: Açık, net, veritabanı öğelerine ve
olası koşullara sadık olmalı; ilgisiz bilgilerden kaçınmalıdır.

{FEWSHOT_EXAMPLES}

### Görev:
Orijinal soru, veritabanı şeması, veritabanı sütun açıklamaları, veritabanı örnekleri, kanıtlar,
olası SQL koşulları **ve zenginleştirilmiş soru adayları ile akıl yürütme listesi** verildiğinde, en
iyi öğeleri entegre eden TEK bir nihai zenginleştirilmiş soru üretin.
Tablo/sütun/değer adlarının veritabanıyla tam olarak eşleştiğinden emin olun (büyük/küçük harf
kullanımına uyun) ve soruyu kendi kendine yeten, belirsiz olmayan bir yapıda tutun.
Sağlanan şemada/açıklamalarda/örneklerde/koşullarda bulunmayan tablolar, sütunlar veya değerler icat
etmeyin.
Yanıt olarak **yalnızca tek bir JSON nesnesi** döndürün ve **başka hiçbir metin eklemeyin**.

# Veritabanı Şeması
{SCHEMA}

# Veritabanı Sütun Açıklamaları
{DB_DESCRIPTIONS}

# Veritabanı Örnekleri
{DB_SAMPLES}

# Olası SQL Koşulları (Aday Yüklemler)
{POSSIBLE_CONDITIONS}

# Orijinal Soru
{QUESTION}

# Kanıtlar
{EVIDENCE}

# Zenginleştirilmiş Soru Adayları (Her bir aday zenginleştirilmiş soruyu ve onun gerekçesini/akıl
yürütmesini içerir.)
{ENRICHED_QUESTION_LIST_WITH_REASONING}

### Yanıt JSON şeması (tam olarak bu biçimi çıktılayın):
{"combined_enrichment_reasoning": "Tek bir metin bloğu olarak kısa, adım adım bir akıl yürütme
sağlayın (adımları numaralandırabilirsiniz).",
"combined_enriched_question": "Tüm adaylardan gelen en iyi öğeleri entegre eden ve SQL iyileştirmesi
için hazır olan TEK bir nihai zenginleştirilmiş soru."}
```

B. QE-Seçim (Selection)

```
### Bir soru ile ilgili veritabanı arasındaki bilgileri kusursuz bir şekilde ilişkilendiren mükemmel
bir veri bilimcisisiniz.
Göreviniz orijinal soruyu, veritabanı şemasını, veritabanı sütun açıklamalarını, veritabanı örnek
değerlerini, mevcut kanıtları ve olası SQL koşullarını **birden fazla zenginleştirilmiş soru adayı ve
onların akıl yürütmeleriyle birlikte** okumaktır. Daha sonra, **tüm adayları değerlendirmeli, temel
olarak tek bir en iyi adayı seçmeli ve gerekirse iyileştirerek** orijinal sorunun amacının veritabanı
şeması kullanılarak en kesin, sadık ve eksiksiz çevirisini sunmalısınız.

### Aşağıdaki talimatları izleyin (QE spesifikasyonuna uyarlanmıştır):
# Adım 1 - Soruyu Dikkatlice Okuyun: Ana amacı ve ayrıntıları anlayın.
# Adım 2 - Veritabanı Şemasını Analiz Edin: İlgili tabloları, sütunları ve ilişkileri belirleyin.
# Adım 3 - Adayların Bağlantılarını İnceleyin: Her bir adayda atıfta bulunulan tabloları, sütunları
ve değerleri şemaya, açıklamalara, örneklere, kanıtlara ve olası SQL koşullarına karşı karşılaştırın.
# Adım 4 - Bütünlüğü ve Kesinliği Değerlendirin: **Orijinal sorunun amacını** veritabanı öğeleriyle
en net, en eksiksiz ve en doğru şekilde eşleştiren adayı belirleyin.
# Adım 5 - En İyi Adayı Seçin ve İyileştirin: Temel olarak tek bir zenginleştirilmiş soruyu seçin.
Orijinal Sorunun amacıyla veya şemayla tam olarak hizalanması için küçük değişiklikler gerekliyse
(örneğin bir büyük/küçük harf hatasını düzeltmek, gereksizliği gidermek), bu değişikliği uygulayın.
Ortaya çıkan soru, sonraki SQL üretimi için belirsizliği en aza indirmeli ve doğruluğu en üst düzeye ç
ıkarmalıdır.
# Adım 6 - Nihai, İyileştirilmiş Soruyu ve seçilen adayın Orijinal Akıl Yürütmesini döndürün.

{FEWSHOT_EXAMPLES}

### Görev:
Orijinal soru, veritabanı bağlamı ve zenginleştirilmiş soru adaylarının listesi (her biri kendi
gerekçesiyle) verildiğinde, **tek bir en iyi adayı seçin ve gerekirse iyileştirin**. Nihai,
muhtemelen iyileştirilmiş zenginleştirilmiş soruyu ve onun **orijinal düşünce zinciri akıl
yürütmesini** (seçilen temel adayı üreten gerekçe) döndürmelisiniz.

Tablo/sütun/değer adlarının veritabanıyla tam olarak eşleştiğinden emin olun (büyük/küçük harf
kullanımına uyun).
Yanıt olarak **yalnızca tek bir JSON nesnesi** döndürün ve **başka hiçbir metin eklemeyin**.

# Veritabanı Şeması
{SCHEMA}

# Veritabanı Sütun Açıklamaları
{DB_DESCRIPTIONS}

# Veritabanı Örnekleri
{DB_SAMPLES}

# Olası SQL Koşulları (Aday Yüklemler)
{POSSIBLE_CONDITIONS}

# Orijinal Soru
{QUESTION}

# Kanıtlar
{EVIDENCE}

# Zenginleştirilmiş Soru Adayları (Her bir aday zenginleştirilmiş soruyu ve onun gerekçesini/akıl
yürütmesini içerir.)
{ENRICHED_QUESTION_LIST_WITH_REASONING}

### Yanıt JSON şeması (tam olarak bu biçimi çıktılayın):
{"selected_enrichment_reasoning": "Seçilen adayın orijinal zenginleştirme akıl yürütmesi
(değiştirilmemelidir).",
"selected_enriched_question": "Nihai, en iyi zenginleştirilmiş soru (seçilen adayın küçük bir
iyileştirmesi olabilir)."}
```

C. Yinelemeli Zenginleştirici (Iterative Enricher)

```
### Bir soru ile ilgili veritabanı arasındaki bilgileri kusursuz bir şekilde ilişkilendirebilen
mükemmel bir veri bilimcisisiniz.
Amacınız, sağlanan orijinal soruyu, onun zenginleştirilmiş halini (v1), bu zenginleştirmenin
arkasındaki akıl yürütmeyi, ilgili veritabanı şemasını, veritabanı sütun açıklamalarını, kanıtları,
olası SQL koşullarını ve soru zenginleştirme örneklerini analiz ederek inceltilmiş ve iyileştirilmiş
bir zenginleştirilmiş soru (v2) oluşturmaktır.
Zenginleştirilmiş soru v2, soru ile veritabanı şeması arasındaki mantıksal ve yapısal bağlantıyı daha
da güçlendirmeli, onu daha da netleştirmeli, daha temelli ve şemaya daha sadık hale getirmelidir.

### Aşağıdaki talimatları izleyin:
Kimlik kuralı:
- Kesinlikle gerekli bir iyileştirme yoksa, `question_enriched_v2` = v1 olarak **bayt-bayt (birebir)
aynı** ayarlayın. Başka sözcüklerle ifade etmeyin, yeniden biçimlendirmeyin veya boşluklara
dokunmayın.
- Değiştirilmediğinde, v1'in zaten en uygun olduğunu ve nedenini belirten bir
`enrichment_reasoning_v2` yazın.
# Adım 1 - Girdileri İnceleyin: Hem orijinal soruyu hem de onun zenginleştirilmiş versiyonunu (v1)
dikkatlice okuyun. Zenginleştirmenin arkasındaki mantıksal akıl yürütmeyi ve şema öğelerini
(tablolar, sütunlar, değerler) soru amacıyla nasıl birleştirdiğini anlayın. Zayıflıkları, gereksiz
tekrarları veya eksik bağlantıları belirleyin.
# Adım 2 - Veritabanı Şemasını Analiz Edin: Şemayı dikkatlice inceleyin ve zenginleştirilmiş soruda
(v1) kullanılan tüm veritabanı öğelerinin gerçekten var olup olmadığını kontrol edin. Eşleşmeyen
tablo veya sütun adlarını düzeltin ve tüm öğelerin şemada temellendiğinden emin olun.
# Adım 3 - Veritabanı Sütun Açıklamalarını İnceleyin: Zenginleştirilmiş sorudaki belirsiz veya net
olmayan terimleri açıklığa kavuşturmak için sütun açıklamalarını kullanın. Daha iyi sütun
referansları varsa, bunları değiştirin.
# Adım 4 - Veritabanı Örnek Değerlerini Analiz Edin ve Gözlemleyin: Tutarlılık ve doğruluğu sağlamak
için zenginleştirilmiş sorudaki ifadeleri ve sayısal eşikleri gerçek örnek değerlerle karşılaştırın.
# Adım 5 - Kanıtları İnceleyin: Kanıtlar, soruyu odaklamak veya iyileştirmek için ipuçları sağlar.
Zenginleştirilmiş sorunun ilgili kanıt öğelerini uygun şekilde entegre ettiğinden, ilgisiz veya
tekrarlayan ayrıntıları çıkardığından emin olun.
# Adım 6 - Olası SQL Koşullarını Analiz Edin: Zenginleştirilmiş sorudaki SQL benzeri mantığın verilen
koşullarla hizalandığını doğrulayın. Daha iyi bir mantıksal ifade veya koşul varsa, onu ayarlayın.
# Adım 7 - Zenginleştirilmiş Soruyu İyileştirin: Zenginleştirilmiş sorunun netliğini, kesinliğini ve ş
emaya uygunluğunu geliştirin. Sadece gerekliyse genişletin veya yeniden ifade edin. Her tablo ve
sütun referansı tam olarak şemada yazıldığı gibi görünmelidir. Gereksiz tekrarlardan veya ilgisiz
bilgilerden kaçının.
# Adım 8 - Gerekçeli Bir Çıktı Üretin: v1 zenginleştirmesine kıyasla ne gibi iyileştirmeler
yapıldığını ve bunların soruyu neden şemaya daha uygun ve mantıksal olarak daha tutarlı hale
getirdiğini açıklayan net bir akıl yürütme (v2) yazın.

{FEWSHOT_EXAMPLES}

### Görev: Aşağıdaki bilgiler ışığında, daha temelli, doğru ve şemaya sadık güncellenmiş bir sürüm
(v2) üretmek için önceden zenginleştirilmiş soruyu inceltin ve geliştirin.

{SCHEMA}
{DB_DESCRIPTIONS}
{DB_SAMPLES}
{POSSIBLE_CONDITIONS}
{QUESTION}
{ENRICHED_QUESTION_V1}
{ENRICHMENT_REASONING_V1}
{EVIDENCE}

### Lütfen aşağıdaki gibi yapılandırılmış bir JSON nesnesi ile yanıt verin:
{"enrichment_reasoning_v2": "Her bir tablonun, sütunun veya koşulun nasıl düzeltildiğine,
açıklandığına veya güçlendirildiğine atıfta bulunarak, önceki zenginleştirme mantığına (v1) göre
yapılan mantıksal iyileştirmelerin ayrıntılı açıklaması.",
"question_enriched_v2": "Şemaya daha uygun, mantıksal olarak yapılandırılmış ve SQL çevirisine hazır,
inceltilmiş zenginleştirilmiş soru (v2)."}

Adım adım düşünelim ve şema, veritabanı açıklamaları ve kanıtlarla mükemmel uyum sağlarken verilen
zenginleştirilmiş soruyu iyileştirelim.
Yanıtınız olarak yalnızca bir JSON çıktısı verin.
```

D. SQL Seçimi

```
### Görev ###
Uzman bir SQL geliştiricisisiniz. Size doğal dilde bir soru, veritabanı şeması ve soruyu yanıtlamak
için üretilmiş ÜÇ adet aday SQL sorgusu verilmiştir. Göreviniz her üç SQL adayını değerlendirmek ve
EN İYİ olanı seçmektir.

### Veritabanı Şeması ###
Veritabanı şeması aşağıda sunulmuştur:
{SCHEMA}

### Veritabanı İçeriği Örnekleri ###
{DB_SAMPLES}

### Ek Veritabanı Açıklamaları ###
{DB_DESCRIPTIONS}

### Doğal Dilde Soru ###
{QUESTION}

### Kanıtlar ###
{EVIDENCE}

### SQL Adayı 1 ###
SQL Sorgusu:
{SQL_CANDIDATE_1}

Akıl Yürütme:
{REASONING_1}

Çalıştırma Durumu: {EXEC_STATUS_1}
{EXEC_INFO_1}

### SQL Adayı 2 ###
SQL Sorgusu:
{SQL_CANDIDATE_2}

Akıl Yürütme:
{REASONING_2}

Çalıştırma Durumu: {EXEC_STATUS_2}
{EXEC_INFO_2}

### SQL Adayı 3 ###
SQL Sorgusu:
{SQL_CANDIDATE_3}

Akıl Yürütme:
{REASONING_3}

Çalıştırma Durumu: {EXEC_STATUS_3}
{EXEC_INFO_3}

### Değerlendirme Kriterleri ###
En iyi SQL sorgusunu seçerken şunları göz önünde bulundurun:
1. **Doğruluk**: SQL doğal dil sorusunu doğru yanıtlıyor mu?
2. **Çalıştırma Başarısı**: SQL hatasız yürütülüyor mu?
3. **Mantıksal Tutarlılık**: SQL mantığı sağlam mı (uygun birleştirmeler, filtreler, toplamalar)?
4. **Bütünlük**: Sorudaki tüm gerekli koşulları ve gereksinimleri içeriyor mu?
5. **Verimlilik**: Doğru sorgular arasında daha basit ve daha verimli olanları tercih edin
6. **Sonuç Geçerliliği**: Eğer çalışma sonuçları sağlanmışsa, mantıklı görünüyorlar mı?

### Talimatlar ###
1. Her SQL adayını dikkatlice analiz edin
2. Bunları soru gereksinimlerine göre karşılaştırın
3. Hem SQL mantığını hem de çalışma sonuçlarını dikkate alın
4. EN İYİ SQL adayını (1, 2 veya 3) seçin
5. Seçiminiz için akıl yürütmenizi sağlayın


### Çıktı Formatı ###
SADECE aşağıdaki formatta geçerli bir JSON nesnesi ile yanıt verin:
```

```
{"chain_of_thought_reasoning": "Tüm üç SQL adayını karşılaştıran ve seçilenin neden en iyisi olduğunu
açıklayan ayrıntılı analiziniz",
 "selected_candidate_index": 1,
 "selected_sql": "Seçilen adayın tam SQL sorgusu"}
```